

A person wearing headphones is working on a laptop. The image is framed by a large white circle with a thin black border. The background is a blurred office or home workspace.

Programación Orientada a Objetos

Capacitador: Ing. Aline Morales

Temas de la primera Semana

Dia 1

Que es un paradigma de programación.

Clase y Objeto

Encapsulamiento

Dia 2

Interface java

CRUD

Dia 3

HashMap

Clase Object

Dia 4

Herencia

Clase Abstrata

Dia 5

Polimorfismo

Expresión Lambda

Dia 1



¿Qué es un lenguaje de programación?

- Un lenguaje de programación es como un idioma que usas para hablar con la computadora y decirle qué hacer

¿Qué es un paradigma de programación?

- Un **paradigma de programación** es un enfoque o estilo que define cómo se deben estructurar y organizar los programas. Es un conjunto de principios y prácticas que guían la forma en que los desarrolladores diseñan e implementan soluciones en un lenguaje de programación.
- **Programación Orientada a Objetos (POO):** Organiza el código en objetos que representan entidades del mundo real.



¿Por qué usar Programación Orientada a Objetos?



Mantenimiento fácil: Al estar organizado en clases y objetos, modificar el código es más sencillo sin afectar otras partes del sistema.



Reutilización del código: La **herencia** permite que una clase reutilice código de otra sin necesidad de copiarlo.



Escalabilidad y modularidad: Permite trabajar en módulos separados sin que el código se vuelva caótico. Facilita el trabajo en equipo en proyectos grandes.



Mayor seguridad: Gracias al **encapsulamiento**, los datos están protegidos y no pueden ser modificados directamente.



Facilidad para representar el mundo real: En POO se pueden modelar entidades como Usuario, Producto o Empleado, haciendo el código más intuitivo.

CLASE



OBJETO



OBJETO



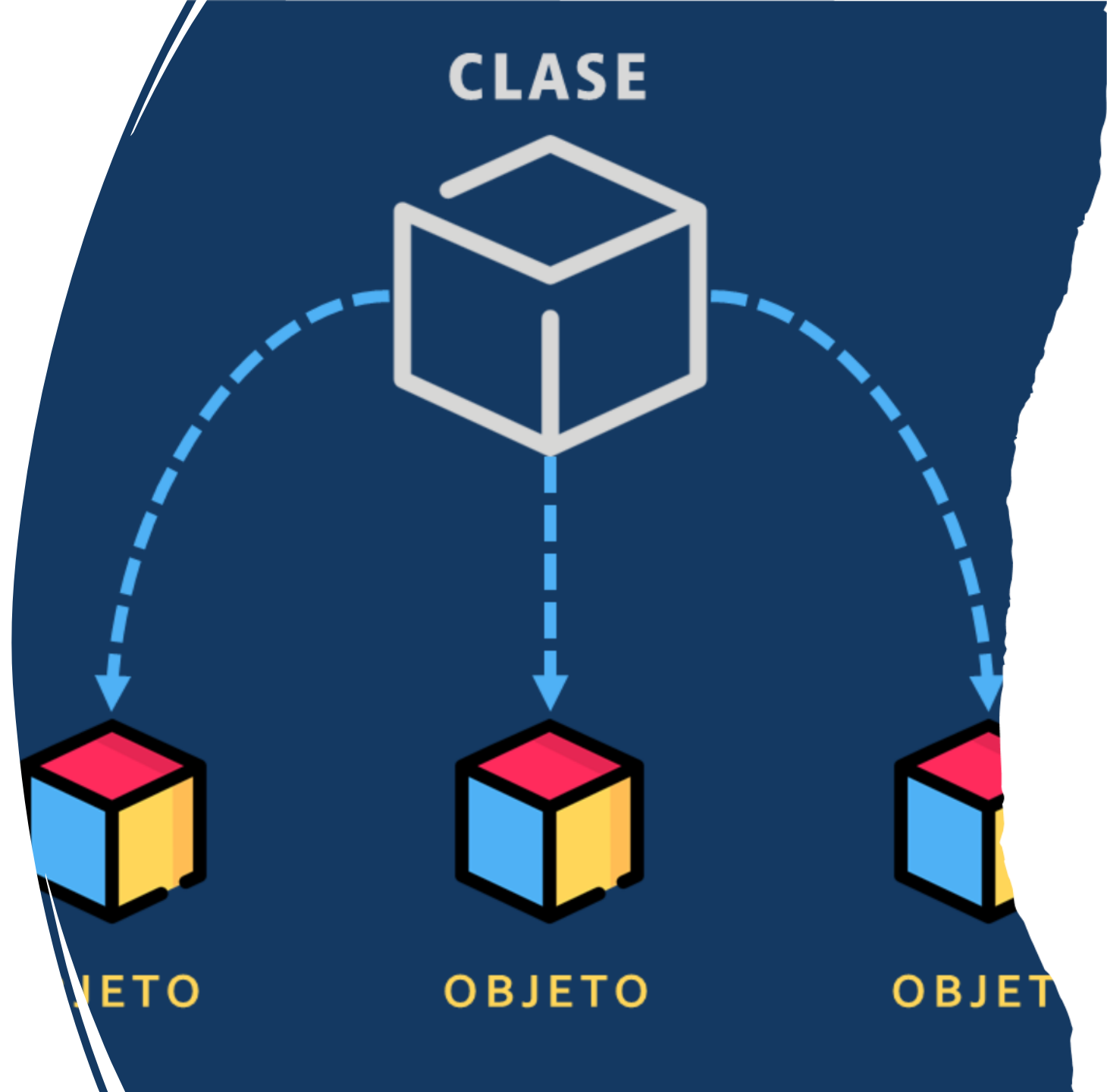
OBJETO

¿Qué es una Clase?

- Una clase es una plantilla o un molde a partir del cual se crean objetos. Define:
- Atributos (propiedades o características) → Son las variables que representan el estado del objeto.
- Métodos (comportamientos o acciones) → Son las funciones que describen el comportamiento del objeto.

¿Qué es un Objeto?

- Un **objeto** es una instancia de una clase, es decir, un ejemplar real creado a partir del molde definido por la clase. Cada objeto tiene su propio estado y puede ejecutar los métodos definidos en la clase.



¿Qué es el Encapsulamiento?

- El **encapsulamiento** es un principio de la Programación Orientada a Objetos que **protege los datos de un objeto y restringe el acceso directo a sus atributos**, permitiendo modificarlos solo a través de métodos controlados.
- Evita la manipulación incorrecta de los datos.
- Aumenta la seguridad del código.
- Permite controlar cómo se accede y modifica la información.
- Facilita el mantenimiento y la escalabilidad del sistema.



Modificadores de Acceso en Java

En Java, podemos restringir el acceso a los atributos y métodos de una clase usando los siguientes modificadores:

Modificador	Accesible dentro de la misma clase	Accesible desde otras clases en el mismo paquete	Accesible desde subclases (herencia)	Accesible desde cualquier clase
private	✓ Sí	✗ No	✗ No	✗ No
default (sin modificador)	✓ Sí	✓ Sí	✗ No	✗ No
protected	✓ Sí	✓ Sí	✓ Sí	✗ No
public	✓ Sí	✓ Sí	✓ Sí	✓ Sí

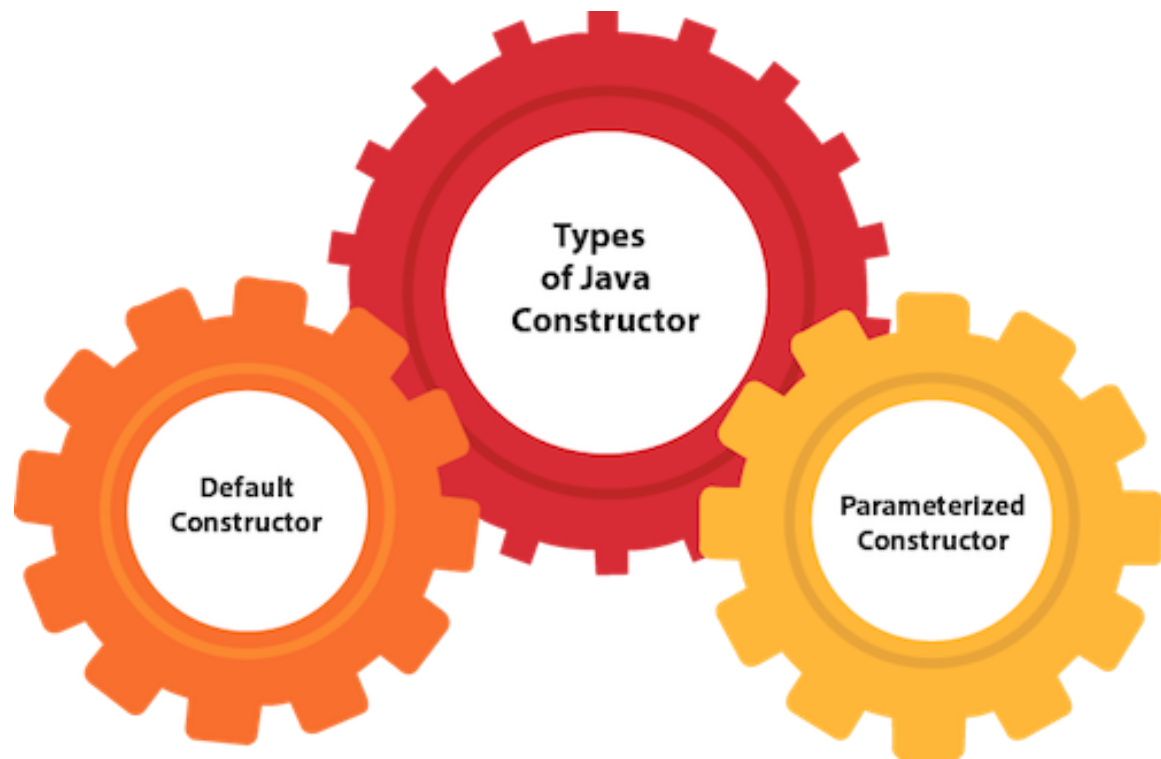
getters y setters

Getters y Setters

Dado que los atributos suelen declararse como private para protegerlos, se utilizan métodos especiales llamados "getters" y "setters" para acceder y modificar estos atributos de forma controlada.

- Getters: Permiten obtener el valor de un atributo.
- Setters: Permiten modificar el valor de un atributo con validaciones.

```
class Persona {  
    constructor(nombre) {  
        this._nombre = nombre;  
    }  
    get nombre() {  
        return this._nombre;  
    }  
    set nombre(nuevoNombre) {  
        this._nombre = nuevoNombre;  
    }  
}
```



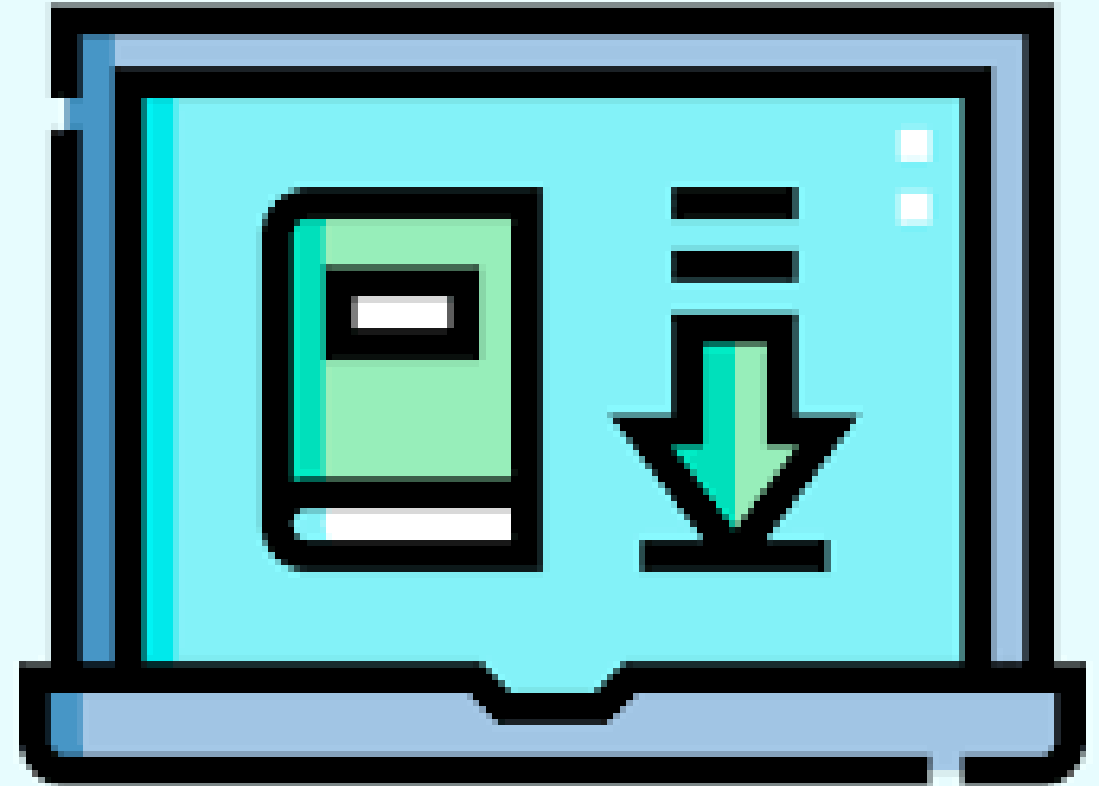
¿Qué es un Constructor?

Un constructor es un método especial de una clase que se ejecuta automáticamente cuando se crea un objeto. Su función principal es inicializar los atributos del objeto. Características del Constructor:

- Tiene el mismo nombre que la clase.
- No tiene un tipo de retorno (ni siquiera void)
- Se ejecuta automáticamente al crear un objeto con new.
- Puede recibir parámetros para inicializar atributos.

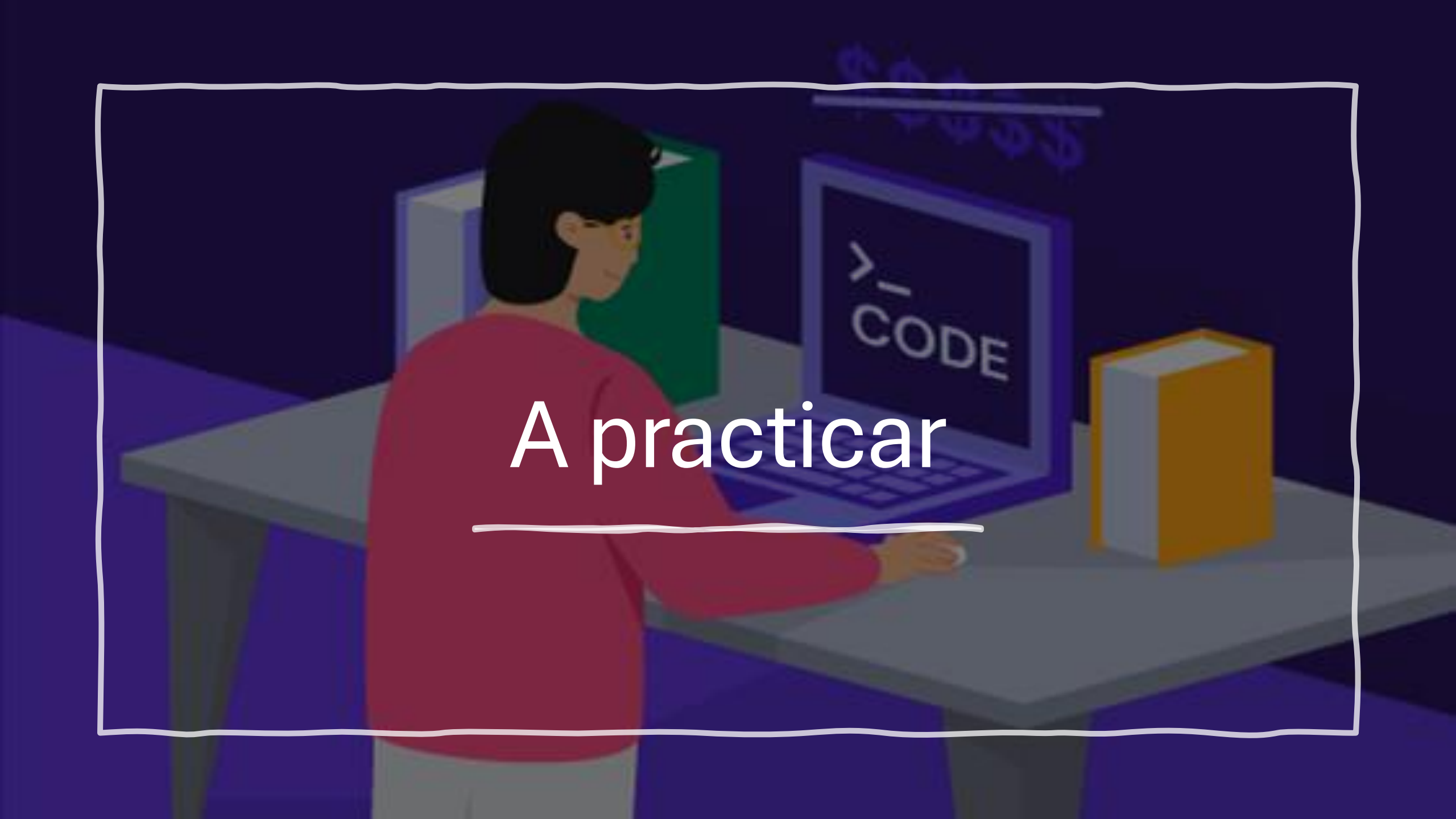
¿Qué es una Lista en Java?

- Una lista es una estructura de datos que permite almacenar elementos de manera ordenada y dinámica. A diferencia de los arreglos (arrays), las listas pueden cambiar de tamaño de forma automática.
- En Java, las listas se manejan con la interfaz List, que pertenece al framework de colecciones de Java (java.util). Las implementaciones más utilizadas son
 - ArrayList → Basada en un array dinámico, rápida en búsquedas.
 - LinkedList → Basada en una lista enlazada, rápida en inserciones y eliminaciones.



Métodos Principales de ArrayList

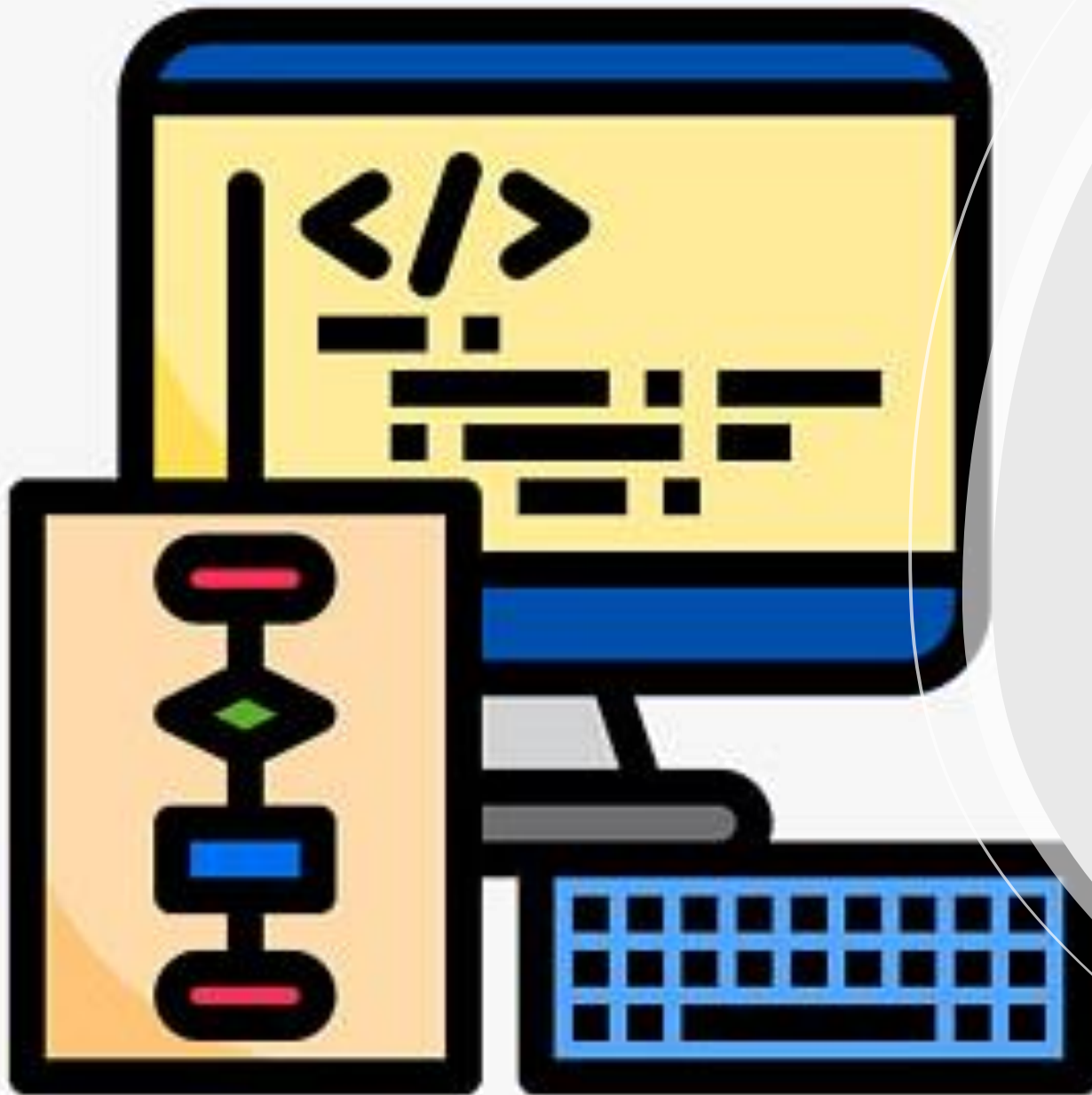
Método	Descripción
<code>add(E elemento)</code>	Agrega un elemento al final.
<code>add(int index, E elemento)</code>	Inserta un elemento en una posición específica.
<code>get(int index)</code>	Obtiene el elemento en la posición indicada.
<code>set(int index, E elemento)</code>	Modifica un elemento en la posición indicada.
<code>remove(int index)</code>	Elimina un elemento por índice.
<code>size()</code>	Devuelve el tamaño de la lista.
<code>contains(E elemento)</code>	Verifica si un elemento está en la lista.
<code>indexOf(E elemento)</code>	Retorna la posición del elemento en la lista.
<code>clear()</code>	Elimina todos los elementos de la lista.

An illustration of a person with short dark hair, wearing a red long-sleeved shirt, sitting at a grey desk. They are looking at a laptop screen that displays the text ">_ CODE". To the right of the laptop is a yellow book. In the background, there are several books on a shelf, including a green one and a blue one. The scene is set in a room with dark blue walls and a dark blue floor. The entire illustration is enclosed in a white, hand-drawn style border.

A praticar

Dia 2





Interface java

- Una **interfaz** es una estructura que declara un conjunto de métodos sin implementar su comportamiento. Cualquier clase que implemente una interfaz debe proporcionar la implementación de esos métodos.

Características Claves de una Interface:

Define un contrato: Obliga a las clases que la implementan a proporcionar ciertos métodos.

No tiene implementación: Solo declara los métodos, sin definir qué hacen.

Permite la abstracción total: Es como un "plano" que las clases deben seguir.

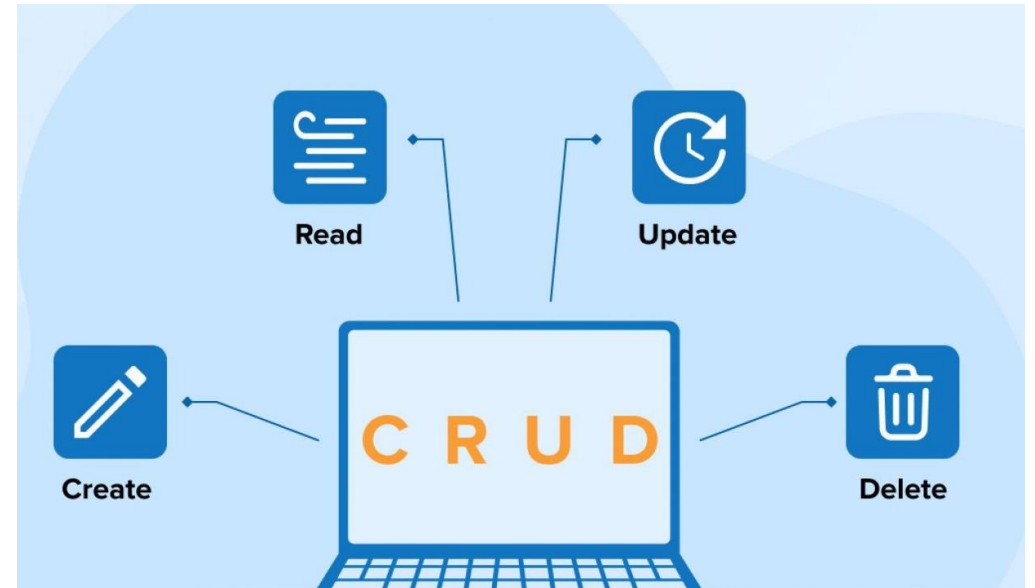
Soporta múltiples implementaciones: Una clase puede implementar múltiples interfaces, a diferencia de la herencia simple en Java.

¿Qué es un Método Abstracto?

- Un **método abstracto** es un método que se declara pero **no se implementa** en una clase abstracta. Es decir, solo se define su firma (nombre, tipo de retorno y parámetros), pero **la implementación la deben proporcionar las clases hijas** que hereden de esa clase abstracta.

¿Qué es un CRUD?

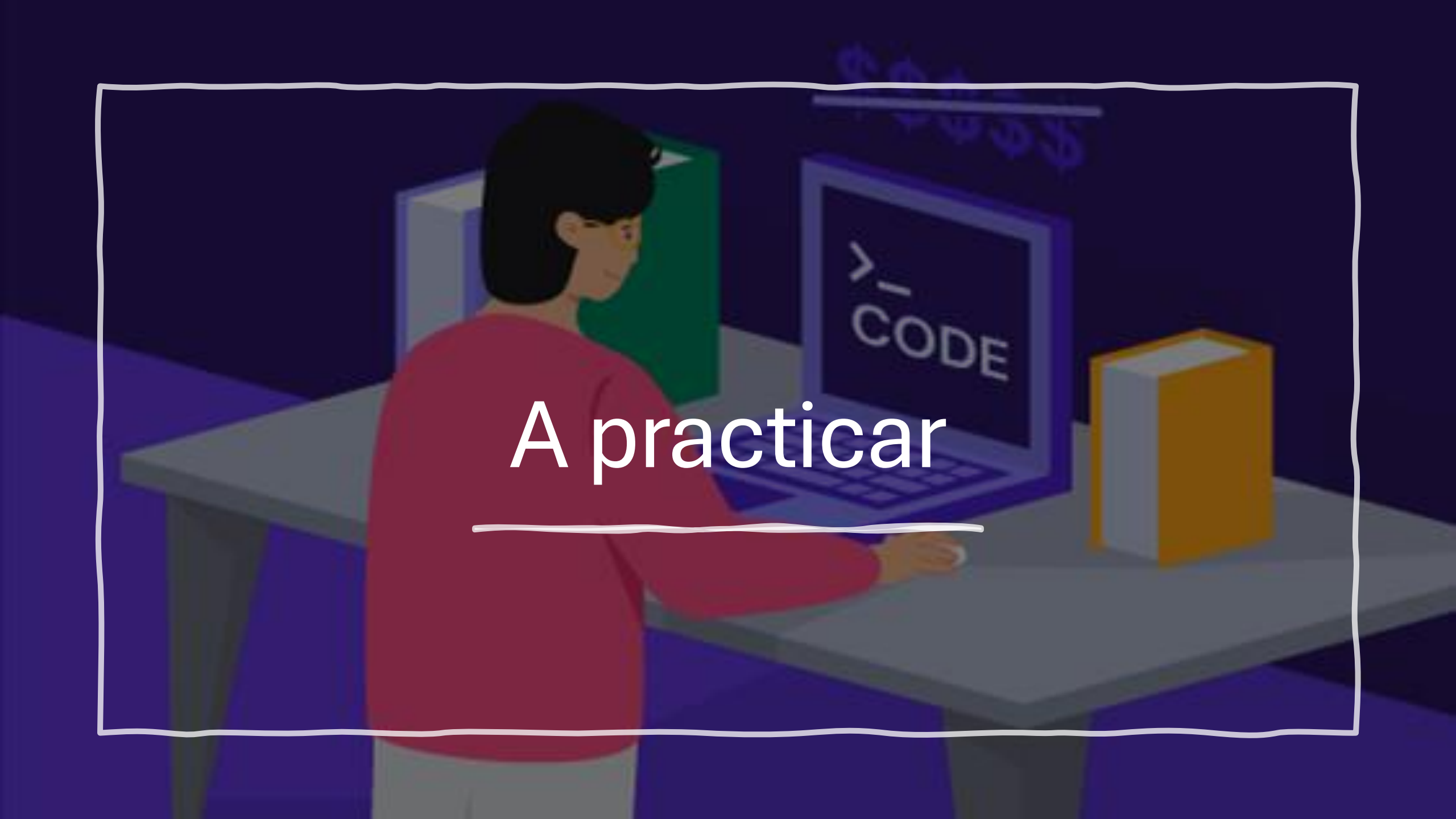
- Un **CRUD** es un conjunto de operaciones básicas que se realizan sobre una base de datos o cualquier tipo de almacenamiento de datos. CRUD significa:



¿Qué es el Manejo de Excepciones en Java?

- El manejo de excepciones en Java permite controlar los errores que pueden ocurrir durante la ejecución de un programa. En lugar de que el programa se detenga abruptamente, podemos capturar y manejar estos errores para evitar fallos inesperados.



An illustration of a person with short dark hair, wearing a red long-sleeved shirt, sitting at a grey desk. They are looking at a laptop screen that displays the text ">_ CODE". To the right of the laptop is a yellow book. In the background, there are several books on a shelf, including a green one and a blue one. The scene is set in a room with dark blue walls and a dark blue floor. The entire illustration is enclosed in a white, hand-drawn style border.

A praticar

Dia 3





HashMap

- Un HashMap en Java es una estructura de datos que implementa la interfaz `Map<K, V>`, permitiendo almacenar y gestionar pares clave-valor. Es parte del paquete `java.util` y se basa en una tabla hash para proporcionar una búsqueda eficiente.

Características principales de HashMap:

Almacena pares clave-valor (K, V).

No permite claves duplicadas, pero sí valores repetidos.

No garantiza un orden específico en los elementos.

Permite valores nulos (null) tanto en claves como en valores.

Es más rápido que TreeMap para operaciones de inserción y búsqueda, debido a su implementación basada en hashing.

No es sincronizado, lo que significa que no es seguro en entornos con múltiples hilos (para eso se usa ConcurrentHashMap).

operaciones útiles

Método	Descripción
put(K clave, V valor)	Agrega o reemplaza un par clave-valor.
get(K clave)	Obtiene el valor asociado a la clave.
remove(K clave)	Elimina el par clave-valor correspondiente.
containsKey(K clave)	Verifica si la clave existe.
containsValue(V valor)	Verifica si un valor existe en el mapa.
size()	Devuelve la cantidad de elementos en el HashMap.
keySet()	Devuelve un conjunto con todas las claves.
values()	Devuelve una colección con todos los valores.
entrySet()	Devuelve un conjunto con las entradas (pares clave-valor).

Tipos de datos primitivos y clases contenedoras

Tipo primitivo	Clase contenedora
<code>boolean</code>	<code>Boolean</code>
<code>char</code>	<code>Character</code>
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>

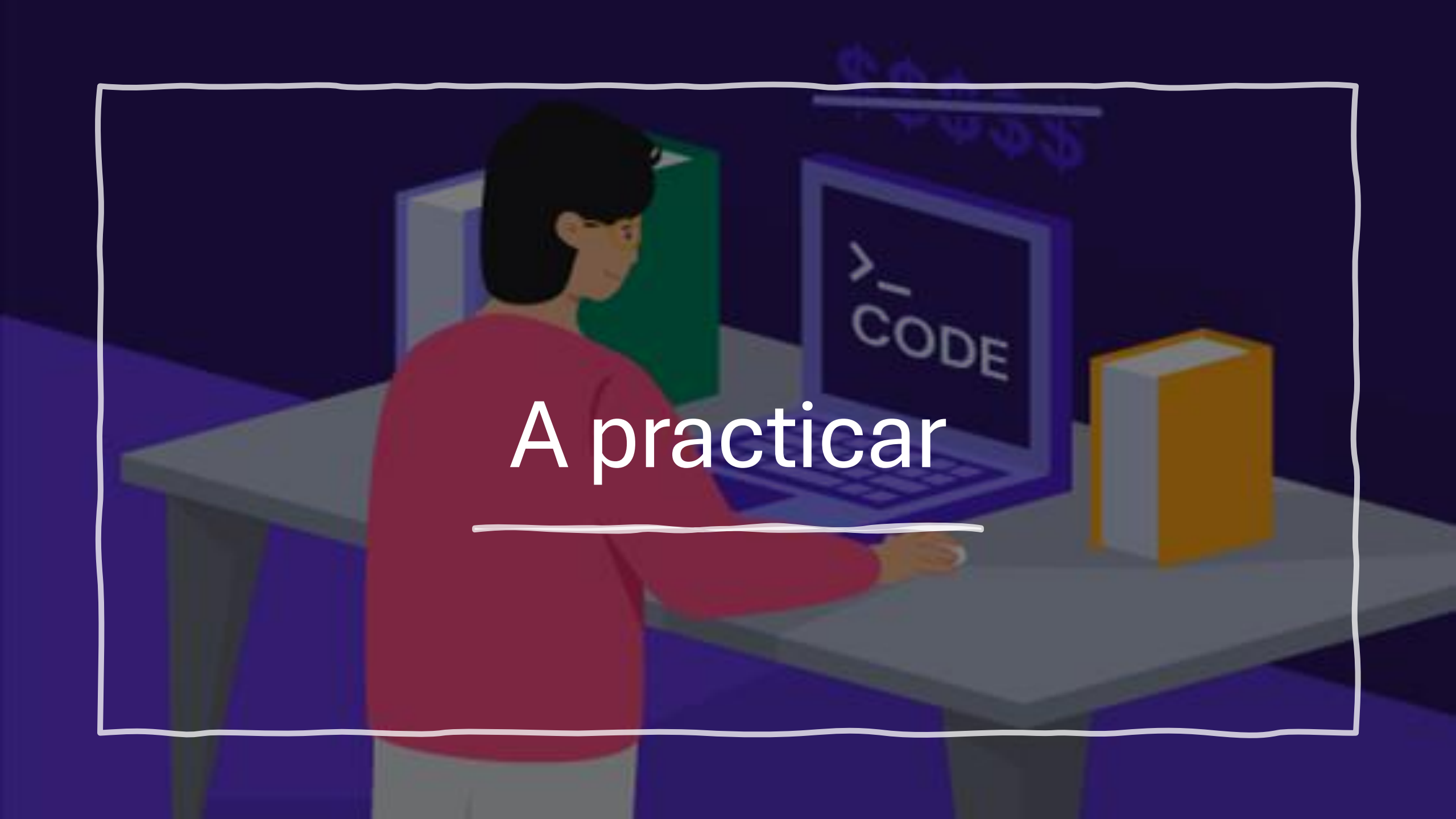
Clase Object en Java

- La clase Object es la superclase de todas las clases en Java. Todas las clases en Java heredan implícitamente de Object, lo que significa que cualquier clase que crees tendrá los métodos de Object por defecto, a menos que los sobrescribas.



Métodos principales de la clase Object

Método	Descripción
<code>equals(Object obj)</code>	Compara si dos objetos son iguales.
<code>hashCode()</code>	Devuelve un código hash del objeto (usado en estructuras como HashMap).
<code>toString()</code>	Devuelve una representación en cadena del objeto.
<code>getClass()</code>	Devuelve el objeto Class del objeto en ejecución.
<code>clone()</code>	Crea una copia del objeto (requiere implementar Cloneable).
<code>finalize()</code>	Se ejecuta antes de que un objeto sea eliminado por el garbage collector .
<code>wait()</code> , <code>notify()</code> , <code>notifyAll()</code>	Métodos para la sincronización de hilos.

An illustration of a person with short dark hair, wearing a red long-sleeved shirt, sitting at a grey desk. They are looking at a laptop screen that displays the text ">_ CODE". To the right of the laptop is a yellow book. In the background, there are several books on a shelf, including a green one and a blue one. The scene is set in a room with dark blue walls and a dark blue floor. The entire illustration is enclosed in a white, hand-drawn style border.

A praticar

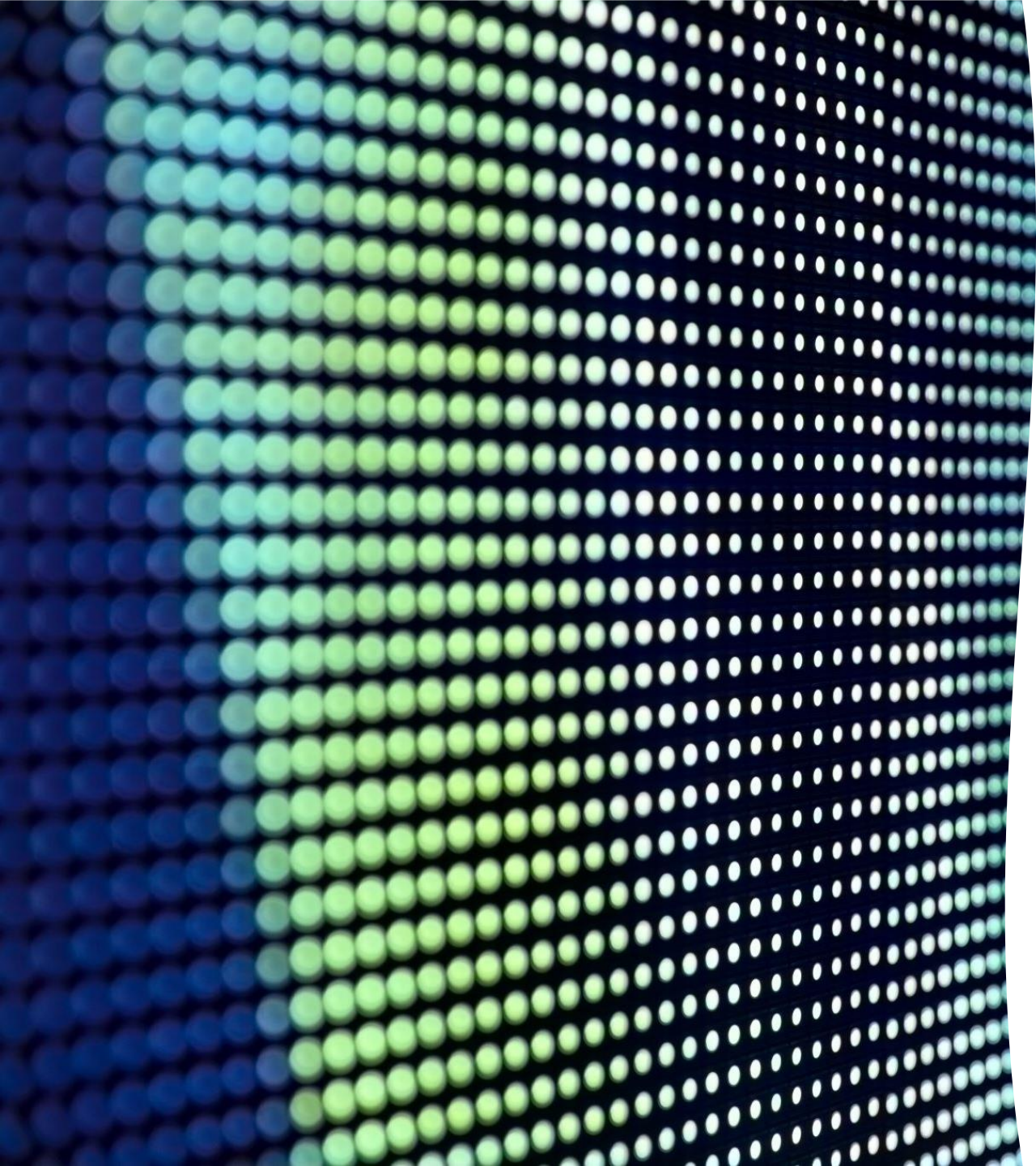
Dia 4





Herencia

- La herencia es un concepto fundamental de la programación orientada a objetos (POO), Su propósito principal es permitir la reutilización de código y establecer una jerarquía entre clases, donde una subclase hereda características (atributos y métodos) de una superclase. Esto evita la duplicación de código y facilita la extensibilidad del software.



Principales características de la herencia

Permite reutilizar código en lugar de escribirlo desde cero.

Establece una relación "es un" (por ejemplo, un Perro es un Animal).

Facilita la extensión y modificación de funcionalidades sin cambiar el código original.

En Java, se usa la palabra clave `extends` para indicar que una clase hereda de otra.

Sobreescritura de métodos

La sobreescritura de métodos (o overriding) es un concepto de herencia en el que una subclase proporciona una nueva implementación de un método que ya existe en la superclase.

Características clave:

- La firma del método debe ser la misma en la subclase (nombre, parámetros y tipo de retorno).
- Solo aplica cuando hay herencia (la subclase sobrescribe un método de la superclase).
- Usa la anotación `@Override` para indicar que un método está siendo sobrescrito (opcional pero recomendable).
- Permite modificar el comportamiento de la superclase sin cambiar su código original.

A stylized illustration on the left side of the slide. It features a laptop in the center, a tablet to its right, and a smartphone below the tablet. They are all connected by a network of lines and nodes, with some nodes highlighted in white. The background is dark blue with some abstract shapes.

Clase Abstracta

- Una clase abstracta es una clase que no se puede instanciar directamente y sirve como modelo para otras clases. Se utiliza cuando queremos definir un comportamiento general que será compartido por varias subclases, pero dejando que cada una implemente ciertos detalles.

Características de una clase abstracta.

Se declara con la palabra clave `abstract`.

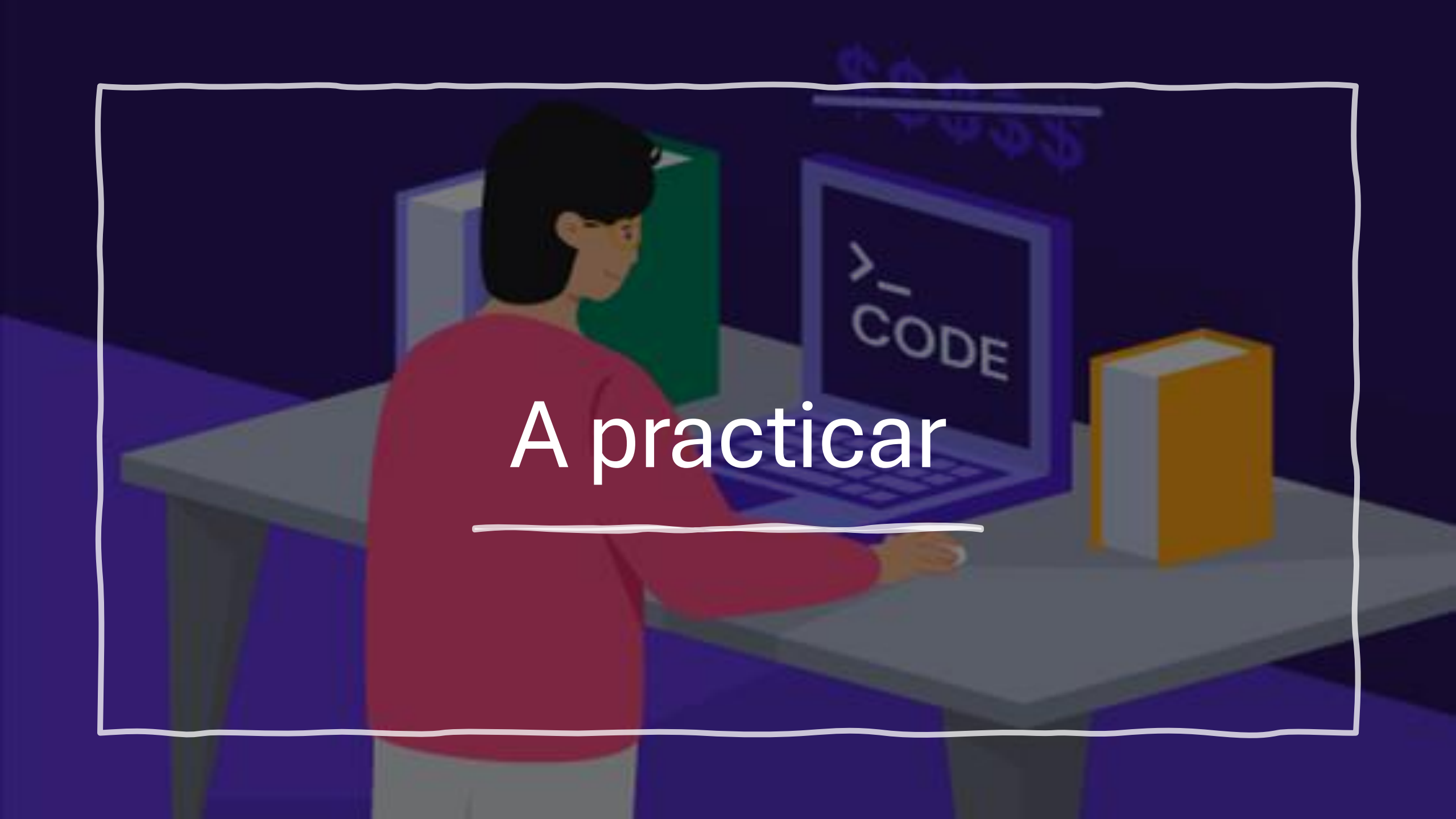
Puede contener métodos abstractos (sin implementación) y métodos concretos (con implementación).

No se puede instanciar directamente, solo se puede heredar.

Las subclases deben implementar los métodos abstractos o también declararse abstractas.

Diferencias Clave entre Clase Abstracta e Interfaz en Java

Característica	Clase Abstracta	Interfaz
Uso	Se usa cuando varias clases comparten características comunes.	Se usa cuando varias clases deben cumplir un contrato sin compartir implementación.
Métodos	Puede tener métodos abstractos (sin implementación) y métodos concretos (con implementación).	Desde Java 8, puede tener métodos abstractos , métodos por defecto (default) y métodos estáticos .
Atributos	Puede tener atributos (variables) con cualquier modificador de acceso (private , protected , public).	Sus atributos son public static final por defecto (constantes).
Herencia/Implementación	Se usa con extends. Una clase solo puede heredar de una clase abstracta (herencia simple).	Se usa con implements. Una clase puede implementar múltiples interfaces (herencia múltiple).
Constructores	Sí puede tener constructores.	No puede tener constructores.
Modificadores de Acceso	Puede tener métodos private , protected y public .	Los métodos son public por defecto.
Ejemplo de Uso	Para representar una jerarquía de clases con comportamiento común.	Para definir un contrato que varias clases deben seguir.

An illustration of a person with short dark hair, wearing a red long-sleeved shirt, sitting at a grey desk. They are looking at a laptop screen that displays the text ">_ CODE". To the right of the laptop is a yellow book. In the background, there are several books on a shelf, including a green one and a blue one. The scene is set in a room with dark blue walls and a dark blue floor. The entire illustration is enclosed in a white, hand-drawn style border.

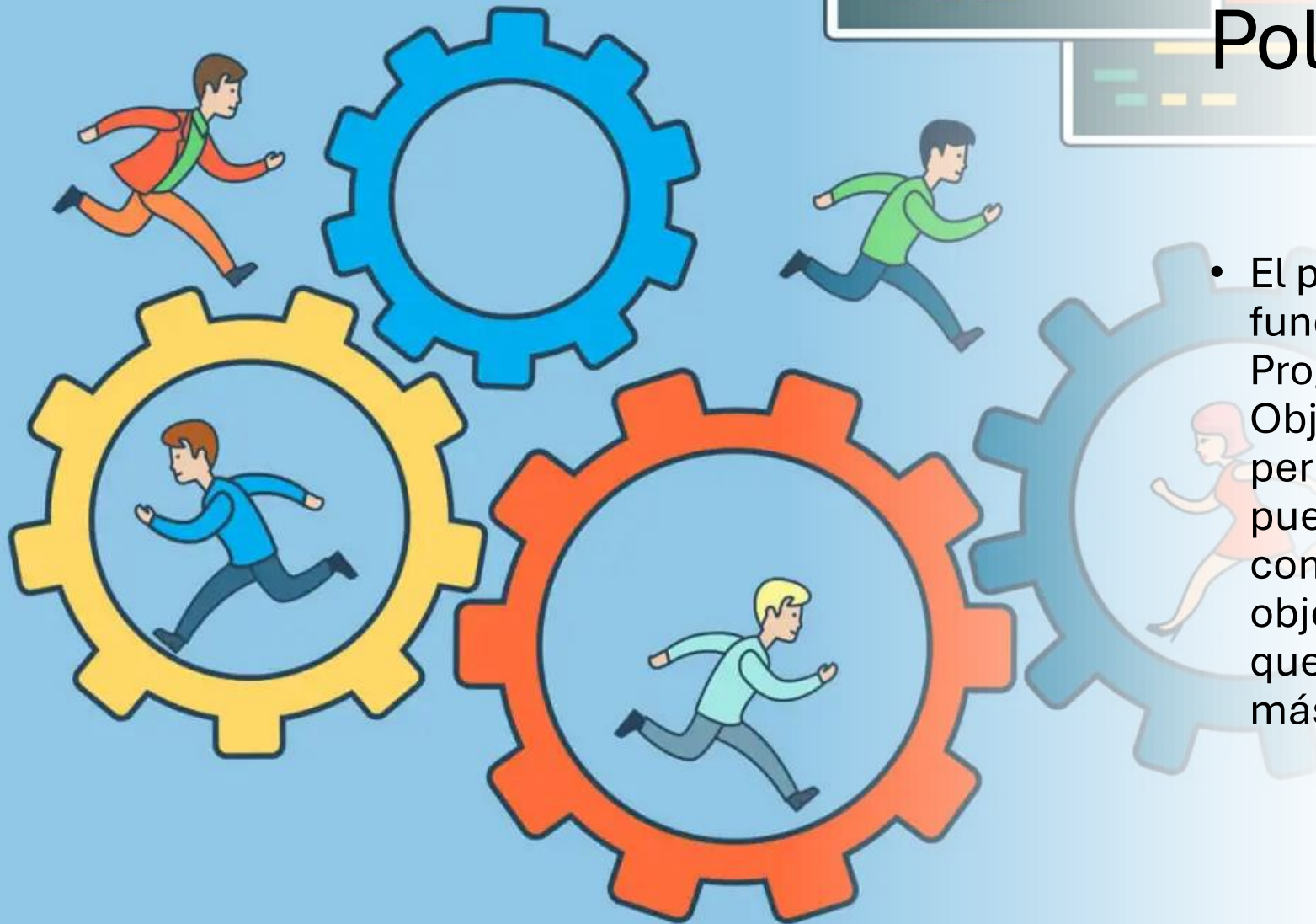
A praticar

Dia 5



Polimorfismo

- El polimorfismo es uno de los fundamentos de la Programación Orientada a Objetos (POO). Este concepto permite que un mismo método pueda tener diferentes comportamientos según el objeto que lo implemente, lo que hace que el código sea más flexible y reutilizable.



Tipos de polimorfismo:

Polimorfismo en tiempo de compilación (Sobrecarga de métodos) Se da cuando en una misma clase existen varios métodos con el mismo nombre, pero con diferentes parámetros (tipo o cantidad).

Polimorfismo en tiempo de ejecución (Sobrescritura de métodos) Se da cuando una clase hija redefine el comportamiento de un método heredado de la clase padre.

Miembros estáticos

Los miembros estáticos se declaran con la palabra clave static y pertenecen a la clase en lugar de a una instancia específica.

¿Qué significa esto?

- Se comparten entre todas las instancias de la clase.
- Solo existe una copia del miembro para toda la clase.
- Se accede a ellos directamente usando el nombre de la clase, sin necesidad de crear un objeto.

Constantes

Las constantes se declaran con la palabra clave final y su valor no puede cambiar después de ser asignado.

¿Qué significa esto?

- Se usan para valores que deben permanecer fijos durante la ejecución del programa.
- Por convención, se escriben en mayúsculas.
- Se pueden combinar con static para que la constante sea compartida entre todas las instancias.

Expresiones Lambda

Las expresiones lambda son una característica introducida en Java 8 que permiten escribir funciones de forma más compacta y clara. Se usan principalmente para trabajar con interfaces funcionales, facilitando la programación funcional en Java.

- Requiere una interfaz funcional (una interfaz con un solo método abstracto).
- Código más corto y legible.
- Se usa comúnmente en Streams y expresiones funcionales.

Las expresiones lambda permiten escribir código más claro y conciso, especialmente en programación funcional y con colecciones. Se usan mucho en Streams, hilos y colecciones.

